

Principles of Programming Languages

Paradigms: Object Oriented Programming.

Andrei Arusoaie¹

¹Department of Computer Science

Outline

Paradigms

Outline

Paradigms

Object-Oriented Programming

Outline

Paradigms

Object-Oriented Programming

Main concepts

Outline

Paradigms

Object-Oriented Programming

Main concepts

Principles (SOLID)

Outline

Paradigms

Object-Oriented Programming

Main concepts

Principles (SOLID)

Design Patterns

Outline

Paradigms

Object-Oriented Programming

Main concepts

Principles (SOLID)

Design Patterns

Conclusion

Paradigms

- ▶ Imperative Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming
- ▶ Declarative programming
 - ▶ Functional Programming

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming
- ▶ Declarative programming
 - ▶ Functional Programming
 - ▶ Logic Programming

Object-Oriented Programming: Motivation-1

- ▶ **Complex** software

Object-Oriented Programming: Motivation-1

- ▶ **Complex** software
- ▶ NATO conference, 1968: *software crisis*
 1. User dissatisfaction with the final product

Object-Oriented Programming: Motivation-1

- ▶ **Complex** software
- ▶ NATO conference, 1968: *software crisis*
 1. User dissatisfaction with the final product
 2. cost overruns

Object-Oriented Programming: Motivation-1

- ▶ **Complex** software
- ▶ NATO conference, 1968: *software crisis*
 1. User dissatisfaction with the final product
 2. cost overruns
 3. buggy software

Object-Oriented Programming: Motivation-1

- ▶ **Complex** software
- ▶ NATO conference, 1968: *software crisis*
 1. User dissatisfaction with the final product
 2. cost overruns
 3. buggy software
 4. brittle software

Motivation-2

Main issues in large software projects:

- ▶ Names: functions, variables, ...

Motivation-2

Main issues in large software projects:

- ▶ Names: functions, variables, ...
- ▶ Constraints: time and space complexity

Motivation-2

Main issues in large software projects:

- ▶ Names: functions, variables, ...
- ▶ Constraints: time and space complexity
- ▶ Memory management (garbage collection and address spaces)

Motivation-2

Main issues in large software projects:

- ▶ Names: functions, variables, ...
- ▶ Constraints: time and space complexity
- ▶ Memory management (garbage collection and address spaces)
- ▶ Concurrency

Motivation-2

Main issues in large software projects:

- ▶ Names: functions, variables, ...
- ▶ Constraints: time and space complexity
- ▶ Memory management (garbage collection and address spaces)
- ▶ Concurrency
- ▶ Event-driven interfaces

Motivation-2

Main issues in large software projects:

- ▶ Names: functions, variables, ...
- ▶ Constraints: time and space complexity
- ▶ Memory management (garbage collection and address spaces)
- ▶ Concurrency
- ▶ Event-driven interfaces

How to fix all these?

How to fix all these?

- ▶ Extreme discipline

How to fix all these?

- ▶ Extreme discipline (hard to get when working in teams)

How to fix all these?

- ▶ Extreme discipline (hard to get when working in teams)
- ▶ Or a new paradigm that facilitates:

How to fix all these?

- ▶ Extreme discipline (hard to get when working in teams)
- ▶ Or a new paradigm that facilitates:
 1. solving very complex problems

How to fix all these?

- ▶ Extreme discipline (hard to get when working in teams)
- ▶ Or a new paradigm that facilitates:
 1. solving very complex problems
 2. working in teams

How to fix all these?

- ▶ Extreme discipline (hard to get when working in teams)
- ▶ Or a new paradigm that facilitates:
 1. solving very complex problems
 2. working in teams
 3. code modularity

How to fix all these?

- ▶ Extreme discipline (hard to get when working in teams)
- ▶ Or a new paradigm that facilitates:
 1. solving very complex problems
 2. working in teams
 3. code modularity
 4. easy development and code maintenance

Object-Oriented Programming

A bunch of concepts:

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors
- ▶ Overloading, overriding, shadowing

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors
- ▶ Overloading, overriding, shadowing
- ▶ *Encapsulation*, information hiding

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors
- ▶ Overloading, overriding, shadowing
- ▶ *Encapsulation*, information hiding
- ▶ *Abstraction*

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors
- ▶ Overloading, overriding, shadowing
- ▶ *Encapsulation*, information hiding
- ▶ *Abstraction*
- ▶ *Inheritance*

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors
- ▶ Overloading, overriding, shadowing
- ▶ *Encapsulation*, information hiding
- ▶ *Abstraction*
- ▶ *Inheritance*
- ▶ *Polymorphism*

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors
- ▶ Overloading, overriding, shadowing
- ▶ *Encapsulation*, information hiding
- ▶ *Abstraction*
- ▶ *Inheritance*
- ▶ *Polymorphism*
- ▶ Dynamic method selection (or dynamic dispatch)

Object-Oriented Programming

A bunch of concepts:

- ▶ Classes, objects, members (attributes + methods), access modifiers
- ▶ Constructors, destructors
- ▶ Overloading, overriding, shadowing
- ▶ *Encapsulation*, information hiding
- ▶ *Abstraction*
- ▶ *Inheritance*
- ▶ *Polymorphism*
- ▶ Dynamic method selection (or dynamic dispatch)
- ▶ Interfaces, abstract classes

Classes vs. Objects

► Class =

Classes vs. Objects

- ▶ Class = a **specification** used to *construct* objects

Classes vs. Objects

- ▶ Class = a **specification** used to *construct* objects
- ▶ Classes establish the name of the class, the *data method* members with types and signature, visibility and implementation.

Classes vs. Objects

- ▶ Class = a **specification** used to *construct* objects
- ▶ Classes establish the name of the class, the *data method* members with types and signature, visibility and implementation.
- ▶ Object =

Classes vs. Objects

- ▶ Class = a **specification** used to *construct* objects
- ▶ Classes establish the name of the class, the *data method* members with types and signature, visibility and implementation.
- ▶ Object = an *instance* of a class; it has a state where its *data members* (or *attributes* or *properties*) – described in the corresponding class – have values

Constructors and Destructors

- ▶ *Constructor*

Constructors and Destructors

- ▶ *Constructor*: a special “method” which has the same name as the class and no return type

Constructors and Destructors

- ▶ *Constructor*: a special “method” which has the same name as the class and no return type
- ▶ When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)

Constructors and Destructors

- ▶ *Constructor*: a special “method” which has the same name as the class and no return type
- ▶ When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- ▶ Typically (in many OO programming languages) we use the `new` keyword to call the constructor

Constructors and Destructors

- ▶ *Constructor*: a special “method” which has the same name as the class and no return type
- ▶ When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- ▶ Typically (in many OO programming languages) we use the `new` keyword to call the constructor
- ▶ *Destructor*

Constructors and Destructors

- ▶ *Constructor*: a special “method” which has the same name as the class and no return type
- ▶ When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- ▶ Typically (in many OO programming languages) we use the `new` keyword to call the constructor
- ▶ *Destructor*: frees the memory occupied by the object

Constructors and Destructors

- ▶ *Constructor*: a special “method” which has the same name as the class and no return type
- ▶ When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- ▶ Typically (in many OO programming languages) we use the `new` keyword to call the constructor
- ▶ *Destructor*: frees the memory occupied by the object
- ▶ *Garbage Collector*

Constructors and Destructors

- ▶ *Constructor*: a special “method” which has the same name as the class and no return type
- ▶ When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- ▶ Typically (in many OO programming languages) we use the `new` keyword to call the constructor
- ▶ *Destructor*: frees the memory occupied by the object
- ▶ *Garbage Collector*: automatic memory management (efficient memory allocation, automatically frees the memory, enables memory reuse)

Encapsulation

► *Encapsulation*

Encapsulation

- ▶ *Encapsulation*: organise the data and operations (over that data) in a single unit

Encapsulation

- ▶ *Encapsulation*: organise the data and operations (over that data) in a single unit
- ▶ Advantage: manipulate the objects in an uniform and consistent way using methods

Encapsulation

- ▶ *Encapsulation*: organise the data and operations (over that data) in a single unit
- ▶ Advantage: manipulate the objects in an uniform and consistent way using methods
- ▶ *Information hiding*: separate the interface (i.e., the public information in a class) from the implementation

Encapsulation

- ▶ *Encapsulation*: organise the data and operations (over that data) in a single unit
- ▶ Advantage: manipulate the objects in an uniform and consistent way using methods
- ▶ *Information hiding*: separate the interface (i.e., the public information in a class) from the implementation
- ▶ *Access modifiers*: used to restrict/grant access to object's data

Abstraction

- ▶ It comes in the same “package” with *Encapsulation*

Abstraction

- ▶ It comes in the same “package” with *Encapsulation*
- ▶ Example: driving a car doesn't require knowing the internals of the combustion engine

Abstraction

- ▶ It comes in the same “package” with *Encapsulation*
- ▶ Example: driving a car doesn't require knowing the internals of the combustion engine
- ▶ *Abstraction*: preserve the information that is important in a given context

Abstraction

- ▶ It comes in the same “package” with *Encapsulation*
- ▶ Example: driving a car doesn't require knowing the internals of the combustion engine
- ▶ *Abstraction*: preserve the information that is important in a given context
- ▶ The human mind can operate easier with abstractions

Basic example

```
public class Car {  
    // encapsulation: via access modifiers  
    // abstraction: only interested in brand and price  
    private String brand;  
    private int price;  
  
    // constructor with params  
    public Car(String brand, int price) {  
        this.brand = brand;  
        this.price = price;  
    }  
  
    // don't break encapsulation: fields can only be read  
    public int getPrice() {  
        return price;  
    }  
  
    public String getBrand() {  
        return brand;  
    }  
}
```

Basic example - usage

```
public class Main {  
    public static void main(String[] args) {  
        Car car = new Car("Dacia", 10000);  
        System.out.println(  
            "A " + car.getBrand() + " " +  
            "costs " + car.getPrice() + "."  
        ); // "A Dacia costs 10000."  
    }  
}
```

Inheritance

► *Inheritance*

Inheritance

- ▶ *Inheritance*: reuse the code/features from a previously defined class

Inheritance

- ▶ *Inheritance*: reuse the code/features from a previously defined class
- ▶ Principle: DRY – “Do not repeat yourself”

Inheritance

- ▶ *Inheritance*: reuse the code/features from a previously defined class
- ▶ Principle: DRY – “Do not repeat yourself”
- ▶ If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...

Inheritance

- ▶ *Inheritance*: reuse the code/features from a previously defined class
- ▶ Principle: DRY – “Do not repeat yourself”
- ▶ If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...
- ▶ ... with some exceptions:

Inheritance

- ▶ *Inheritance*: reuse the code/features from a previously defined class
- ▶ Principle: DRY – “Do not repeat yourself”
- ▶ If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...
- ▶ ... with some exceptions:
 - ▶ *Field hiding*: the `Child` class can redefine data members (i.e, use the same name) from the `Parent` class

Inheritance

- ▶ *Inheritance*: reuse the code/features from a previously defined class
- ▶ Principle: DRY – “Do not repeat yourself”
- ▶ If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...
- ▶ ... with some exceptions:
 - ▶ *Field hiding*: the `Child` class can redefine data members (i.e, use the same name) from the `Parent` class
 - ▶ *Overriding*: the `Child` class can override methods defined in `Parent`

Field hiding

```
public class Parent {  
    int someX = 0;  
}  
  
public class Child extends Parent {  
    int someX = 1; // field hiding  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
        Child child = new Child();  
        System.out.println(child.someX); // 1  
  
        Parent pChild = (Parent) child;  
        System.out.println(pChild.someX); // 0  
    }  
}
```

Overriding

```
public class Parent {  
    void foo() {  
        System.out.println("parent foo");  
    }  
}  
  
public class Child extends Parent {  
    @Override  
    void foo() {  
        System.out.println("child foo");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Child child = new Child();  
        child.foo(); // "child foo"  
  
        Parent pChild = (Parent) child;  
        pChild.foo(); // "child foo" due to dynamic binding  
    }  
}
```

Polymorphism

It comes in many flavours:

Polymorphism

It comes in many flavours:

1. Overloading: multiple implementations of the same method with different parameters

Polymorphism

It comes in many flavours:

1. Overloading: multiple implementations of the same method with different parameters
2. Parametric – often used with *generics*: methods work with parametric classes (e.g., `sort` for `List<T>`)

Polymorphism

It comes in many flavours:

1. Overloading: multiple implementations of the same method with different parameters
2. Parametric – often used with *generics*: methods work with parametric classes (e.g., `sort` for `List<T>`)
3. Subtype: single interface to multiple representations of the same type (the most common type of polymorphism)

Overloading - different parameters types

```
public class Arithmetic {  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public double add(double a, double b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        Arithmetic arithmetic = new Arithmetic();  
        System.out.println(arithmetic.add(2, 4)); // 6  
        System.out.println(arithmetic.add(2.5, 3.5)); // 6.0  
    }  
}
```

Overloading - different number of parameters

```
public class Arithmetic {  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public int add(int a, int b, int modulo) {  
        return (a + b) % modulo;  
    }  
  
    public static void main(String[] args) {  
        Arithmetic arithmetic = new Arithmetic();  
        System.out.println(arithmetic.add(2, 4)); // 6  
        System.out.println(arithmetic.add(22, 4, 3)); // 2  
    }  
}
```

Parametric - Generics

```
// this class is parameterised by a type  
public class Wrapper<T> {  
    private T t;  
  
    public void set(T t) {  
        this.t = t;  
    }  
  
    public T get() {  
        return t;  
    }  
}
```

Parametric - Generics: usage

```
public class Main {  
    public static void main(String[] args) {  
        Wrapper<String> stringWrapper = new Wrapper<>();  
        stringWrapper.set("this is a string");  
        System.out.println(stringWrapper.get());  
  
        Wrapper<Integer> integerWrapper = new Wrapper<>();  
        integerWrapper.set(100);  
        System.out.println(integerWrapper.get());  
    }  
}
```

Subtype polymorphism

The Liskov Substitution Principle:

- ▶ “Derived types must be completely substitutable for their base types”
- ▶ The subtype polymorphism is realised via the Liskov Substitution Principle

Polygons: Rectangle

```
public abstract class Polygon {  
    public abstract int getArea();  
}  
  
public class Rectangle extends Polygon {  
    private int width;  
    private int height;  
    public Rectangle(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
    @Override  
    public int getArea() {  
        return width * height;  
    }  
}
```

Polygons: Square

```
public abstract class Polygon {  
    public abstract int getArea();  
}  
  
public class Square extends Polygon {  
    private int side;  
    public Square(int side) {  
        this.side = side;  
    }  
    @Override  
    public int getArea() {  
        return side * side;  
    }  
}
```

LSP: Liskov Substitution Principle applied

```
public class LiskovSubstitutionPrinciple {  
    public void calculateArea(Polygon p) {  
        System.out.println(p.getArea());  
    }  
  
    public static void main(String[] args) {  
        LiskovSubstitutionPrinciple lsp =  
            new LiskovSubstitutionPrinciple();  
  
        // lsp.calculateArea accepts any Polygon as argument  
        Rectangle rectangleNoOCP = new Rectangle(10, 15);  
        lsp.calculateArea(rectangleNoOCP);  
  
        Square square = new Square(10);  
        lsp.calculateArea(square);  
    }  
}
```

LSP violation by example

```
class Rectangle_ {  
    private int width;  
    private int height;  
  
    public void setWidth(int width) {  
        this.width = width;  
    }  
  
    public void setHeight(int height) {  
        this.height = height;  
    }  
  
    public int getArea() {  
        return width * height;  
    }  
}
```

```
// just because square is a particular type of rectangleNoOCP  
class Square_ extends Rectangle_ { }
```

LSP violation by example

```
public class LSPViolation {  
    public static void main(String[] args) {  
        Rectangle_ r = new Rectangle_();  
        r.setWidth(10);  
        r.setHeight(15);  
        System.out.println(r.getArea());  
  
        Square_ s = new Square_();  
        s.setHeight(10); // why is this allowed?  
        s.setWidth(15); // ???  
        System.out.println(s.getArea()); // wrong output  
    }  
}
```

The derived class `Square` cannot be substitutable for the base type `Rectangle`.

Dynamic Dispatch

Selection of the implementation of a (polymorphic) operation (method or function) at run time

```
public class Parent {  
    public void foo() {  
        System.out.println("foo in parent");  
    }  
}  
  
public class Child extends Parent {  
    @Override  
    public void foo() {  
        System.out.println("foo in child");  
    }  
}
```

Dynamic Dispatch

```
public class Main {  
    public static void main(String[] args) {  
        Parent parent = new Parent();  
        parent.foo(); // foo in parent  
  
        Child child = new Child();  
        child.foo(); // foo in child  
        ((Parent) child).foo(); // 'foo in child'  
        // dynamic dispatch: dynamic selection  
        // of the implementation for foo()  
        // based on the actual type of 'child'  
    }  
}
```

Name binding; early vs late binding

- ▶ Name binding: association between identifiers and entities (like data or code)
- ▶ An identifier associated to an object is a *reference* to that object
- ▶ Static (early) binding: name binding is performed at *compile* time
- ▶ Dynamic (late/virtual) binding: name binding is performed at *run* time

Exercising name binding in Java

```
public class NameBinding {  
    public static void main(String[] args) {  
        List<String> list;  
  
        // list is a ref to a linked list  
        list = new LinkedList<String>();  
  
        // the referenced linked list is modified/mutated  
        list.add("foo");  
        System.out.println(list); // [foo]  
  
        // the *same* name is now assigned/bound to null  
        list = null;  
        System.out.println(list); // null  
    }  
}
```

Late binding vs dynamic dispatch

- ▶ Often, late binding and dynamic dispatch are confused
- ▶ Late binding implies dynamic dispatch
- ▶ Dynamic dispatch does not imply late binding
- ▶ Mechanisms to achieve dynamic dispatch:
 1. in typed languages, the type is used to decide to resolve the implementation
 2. dispatch tables: associate pointers or memory addresses to functions or methods

Late binding vs dynamic dispatch

- ▶ Often, late binding and dynamic dispatch are confused
- ▶ Late binding implies dynamic dispatch
- ▶ Dynamic dispatch does not imply late binding
- ▶ Mechanisms to achieve dynamic dispatch:
 1. in typed languages, the type is used to decide to resolve the implementation
 2. dispatch tables: associate pointers or memory addresses to functions or methods
- ▶ There is no late binding in C++, but only dynamic dispatch: virtual tables (vtables) cannot be changed at runtime
- ▶ In Java: the JVM identifies the actual type of the object at *runtime* and resolves the method calls based on that type (not on the type of the reference)

OOP - related concepts

- ▶ Interfaces
- ▶ Abstract Classes
- ▶ SOLID Principles
- ▶ Design Patterns

Interfaces

Interface: a collection of methods without implementation which serves as a contract for the class implementing it.

```
public interface Switchable {  
    void turnOn();  
    void turnOff();  
}  
  
public interface Connectable {  
    void connect();  
    void disconnect();  
}
```

Interfaces implemented by class

```
public class Smartphone implements Switchable, Connectable {  
    @Override  
    public void connect() {  
        System.out.println("Connect smartphone");  
    }  
    @Override  
    public void disconnect() {  
        System.out.println("Disconnect smartphone");  
    }  
    @Override  
    public void turnOn() {  
        System.out.println("Turn on smartphone");  
    }  
    @Override  
    public void turnOff() {  
        System.out.println("Turn off smartphone");  
    }  
}
```

Interfaces

```
public class Main {  
    public static void main(String[] args) {  
        // Creating an instance of SmartDevice  
        Smartphone smartphone = new Smartphone();  
  
        // Operating the device using Switchable interface  
        smartphone.turnOn();  
        smartphone.turnOff();  
  
        // Operating the device using Connectable interface  
        smartphone.connect();  
        smartphone.disconnect();  
    }  
}  
  
// Output:  
// Turn on smartphone  
// Turn off smartphone  
// Connect smartphone  
// Disconnect smartphone
```

Abstract classes

Abstract class: classes that cannot be instantiated on their own and may contain abstract methods.

```
abstract class ElectronicDevice {  
    private String brand;  
    public ElectronicDevice(String brand) {  
        this.brand = brand;  
    }  
    // Abstract method to turn on the device  
    abstract void turnOn();  
    // Abstract method to turn off the device  
    abstract void turnOff();  
    // Non-abstract method with a default implementation  
    void displayBrand() {  
        System.out.println("Brand: " + brand);  
    }  
}
```

Concrete implementation

```
public class Laptop extends ElectronicDevice {
    private String operatingSystem;
    public Laptop(String brand, String operatingSystem) {
        super(brand); // call the abstract class constructor
        this.operatingSystem = operatingSystem; // specific fi
    }
    // Implementation of turnOn for Laptop
    @Override
    void turnOn() {
        System.out.println("Laptop is ON");
    }
    // Implementation of turnOff for Laptop
    @Override
    void turnOff() {
        System.out.println("Laptop is OFF");
    }
}
```

Usage

```
public class AbstractMain {  
    public static void main(String[] args) {  
        Laptop mac = new Laptop("Apple", "Mac OS");  
        mac.turnOn(); // Laptop is ON  
        mac.turnOff(); // Laptop is OFF  
        mac.displayBrand(); // Brand: Apple  
    }  
}
```

Interfaces vs. Abstract Classes

Similarities:

- ▶ Support for polymorphism
- ▶ Methods with no implementation
- ▶ Support for inheritance

Some tasks can be achieved using either an interface or an abstract class.

In C++ interfaces are simulated using abstract classes

Interfaces vs. Abstract Classes

Conceptual differences:

- ▶ Interfaces cannot contain constructors, abstract classes can
- ▶ Interfaces are abstract and public; abstract classes can contain concrete implementations
- ▶ Interfaces can contain only constant (public, static, final); abstract classes can contain instance variables too
- ▶ Interfaces support multiple inheritance; abstract classes do not

Depending on the language implementation, you may find that sometimes these conceptual differences are not present.

Interfaces vs. Abstract Classes

Conceptual differences:

- ▶ Interfaces cannot contain constructors, abstract classes can
- ▶ Interfaces are abstract and public; abstract classes can contain concrete implementations
- ▶ Interfaces can contain only constant (public, static, final); abstract classes can contain instance variables too
- ▶ Interfaces support multiple inheritance; abstract classes do not

Depending on the language implementation, you may find that sometimes these conceptual differences are not present. For example, in Java, you can have default implementations for methods in an interface

Also, in C++, you can have multiple inheritance for abstract classes.

SOLID Principles

- ▶ Single Responsibility Principle (SRP)
- ▶ Open/Closed Principle (OCP)
- ▶ Liskov Substitution Principle (LSP) – already discussed
- ▶ Interface Segregation Principle (ISP)
- ▶ Dependency Inversion Principle (DIP)

Single Responsibility Principle (SRP)

Each class should have only one responsibility, and thus, it should have only one reason to change.

// Violation of the SRP

```
public class Book {
    private String title;
    private String author;
    private int pageCount;
    public Book(String title, String author, int pageCount) {
        this.title = title;
        this.author = author;
        this.pageCount = pageCount;
    }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public int getPageCount() { return pageCount; }
    // Responsibility not related to book representation:
    public String displayDetails() {
        return "Title: " + title + "\nAuthor: " + author +
            "\nPage Count: " + pageCount;
    }
}
```

Single Responsibility Principle (SRP) - correct

```
// Book class responsible for managing book information
public class Book {
    private String title;
    private String author;
    private int pageCount;
    public Book(String title, String author, int pageCount) {
        this.title = title;
        this.author = author;
        this.pageCount = pageCount;
    }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public int getPageCount() { return pageCount; }
}
```

Single Responsibility Principle (SRP) - correct

```
// BookFormatter is responsible for formatting book details
public class BookFormatter {
    public String formatDetails(Book book) {
        return "Title: " + book.getTitle() + "\nAuthor: "
            + book.getAuthor()
            + "\nPage Count: " + book.getPageCount();
    }
}

public class Main {
    public static void main(String[] args) {
        Book book = new Book("The Music of Chance",
            "Paul Auster", 198);

        BookFormatter bookFormatter = new BookFormatter();
        System.out.println(bookFormatter.formatDetails(book));
    }
}
```

Open/Closed Principle (OCP)

Software entities (classes, modules, functions) should be open for extension but closed for modification.

```
public abstract class Shape { }
public class Circle extends Shape {
    private int radius;
    public Circle(int radius) {
        this.radius = radius;
    }
    public int getRadius() { return radius; }
}
public class Rectangle extends Shape {
    private int width;
    private int height;
    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }
    public int getWidth() { return width; }
    public int getHeight() { return height; }
}
```

Open/Closed Principle (OCP)

The class below violates OCP:

```
// The AreaCalculator class violates the OCP
// because every time we add a new shapeNoOCP,
// we need to modify the existing class.
public class AreaCalculator {
    public double calculateArea(Shape shapeNoOCP) {
        if (shapeNoOCP instanceof Circle) {
            Circle circleNoOCP = (Circle) shapeNoOCP;
            return Math.PI *
                Math.pow(circleNoOCP.getRadius(), 2);
        } else if (shapeNoOCP instanceof Rectangle) {
            Rectangle rectangleNoOCP = (Rectangle) shapeNoOCP;
            return rectangleNoOCP.getWidth() *
                rectangleNoOCP.getHeight();
        }
    }
}
```

Open/Closed Principle (OCP) - correct

To adhere to OCP, first create an interface

```
public interface Shape {  
    double calculateArea();  
}
```

Then, implement it:

```
public class Circle implements Shape{  
    private int radius;  
    public Circle(int radius) {  
        this.radius = radius;  
    }  
    public int getRadius() { return radius; }  
  
    @Override  
    public double calculateArea() {  
        return Math.PI * Math.pow(this.getRadius(), 2);  
    }  
}
```

Open/Closed Principle (OCP) - correct

Other implementation:

```
public class Rectangle implements Shape{
    private int width;
    private int height;
    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }
    public int getWidth() { return width; }
    public int getHeight() { return height; }
    @Override
    public double calculateArea() {
        return this.getWidth() * this.getHeight();
    }
}
```

Adding a new shape won't require any code change:

```
public class AreaCalculator {
    public double calculateArea(Shape shape) {
        return shape.calculateArea();
    }
}
```

Open/Closed Principle (OCP) - correct

Adding a new class won't change the existing code at all! We can simply create a new shape and none of the existing code has to be changed or adapted.

```
public class Ellipse implements Shape{
    private double majorAxis;
    private double minorAxis;
    public Ellipse(double majorAxis, double minorAxis) {
        this.majorAxis = majorAxis;
        this.minorAxis = minorAxis;
    }
    public double getMajorAxis() { return majorAxis; }
    public double getMinorAxis() { return minorAxis; }

    @Override
    public double calculateArea() {
        return Math.PI * majorAxis * minorAxis;
    }
}
```

Open/Closed Principle (OCP) - execution

```
public static void main(String[] args) {  
    Circle circle = new Circle(10);  
    Rectangle rectangle = new Rectangle(100, 10);  
    Ellipse ellipse = new Ellipse(10.0, 5.0);  
  
    AreaCalculator areaCalculator = new AreaCalculator();  
    System.out.println(areaCalculator.calculateArea(circle));  
    System.out.println(areaCalculator.calculateArea(rectangle));  
    System.out.println(areaCalculator.calculateArea(ellipse));  
}
```

```
// Output:  
// 314.1592653589793  
// 1000.0  
// 157.07963267948966
```

Interface Segregation Principle (ISP)

Instead of having a single, large interface, it is better to have multiple smaller interfaces tailored to specific use cases. Classes should not be forced to implement certain functions.

```
public interface Bird {  
    void fly();  
    void swim();  
    void eat();  
}  
  
public class Penguin implements Bird{  
    @Override  
    public void fly() {  
        System.out.println("penguins can't fly"); // ouch  
    }  
    @Override  
    public void swim() {  
        System.out.println("penguins are pro swimmers");  
    }  
    @Override  
    public void eat() {  
        System.out.println("penguins eat fish");  
    }  
}
```

Interface Segregation Principle (ISP) - correct

To adhere to ISP, we can split the big interface into more specific interfaces based on functionalities.

```
public interface IEat {  
    void eat();  
}  
  
public interface IFly {  
    void fly();  
}  
  
public interface ISwim {  
    void swim();  
}
```

Interface Segregation Principle (ISP) - correct

Now the main class implements only certain interfaces, without being forced to implement irrelevant methods.

```
public class Penguin implements ISwim, IEat{
    @Override
    public void swim() {
        System.out.println("penguins are pro swimmers");
    }
    @Override
    public void eat() {
        System.out.println("penguins eat fish");
    }
}
```

Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules;
both should depend on abstractions.

Abstractions (interfaces or abstract classes) should not depend
on details; details should depend on abstractions.

```
public class SpellChecker {  
    public void checkSpelling() {  
        System.out.println("spell checking ...");  
    }  
}  
  
public class TextEditor {  
    private SpellChecker spellChecker;  
    // Constructor-based dependency injection  
    public TextEditor() {  
        this.spellChecker = new SpellChecker(); // issue here  
    }  
    public void spellCheck() {  
        spellChecker.checkSpelling();  
    }  
}
```

Dependency Inversion Principle (DIP) - correct

```
public class TextEditor {  
    private SpellChecker spellChecker;  
    // Constructor-based dependency injection  
    public TextEditor(SpellChecker spellChecker) {  
        this.spellChecker = spellChecker;  
    }  
    public void spellCheck() {  
        spellChecker.checkSpelling();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        SpellChecker spellChecker = new SpellChecker();  
        // dependency is passed through constructor  
        TextEditor textEditor = new TextEditor(spellChecker);  
        textEditor.spellCheck();  
    }  
}
```

Design Patterns

- ▶ Design patterns in object-oriented programming (OOP) are reusable solutions to common problems that occur in software design.
- ▶ They represent best practices for solving certain types of problems and provide a way to structure and organize code in a more maintainable and scalable manner.
- ▶ Design patterns are not specific implementations or code snippets; rather, they are general concepts or templates that can be adapted and applied to various situations.

Design Patterns - categories

Design Patterns - categories

Creational Patterns:

- ▶ deal with the process of object creation, providing mechanisms for creating objects in a way that is flexible, efficient, and independent of the system's architecture.
- ▶ Examples: Singleton, Factory Method, Abstract Factory, Builder, and Prototype patterns.

Design Patterns - categories

Creational Patterns:

- ▶ deal with the process of object creation, providing mechanisms for creating objects in a way that is flexible, efficient, and independent of the system's architecture.
- ▶ Examples: Singleton, Factory Method, Abstract Factory, Builder, and Prototype patterns.

Structural Patterns:

- ▶ focus on the composition of classes or objects, helping to form larger structures and interfaces from simpler ones.
- ▶ Examples: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, and Proxy patterns.

Design Patterns - categories

Creational Patterns:

- ▶ deal with the process of object creation, providing mechanisms for creating objects in a way that is flexible, efficient, and independent of the system's architecture.
- ▶ Examples: Singleton, Factory Method, Abstract Factory, Builder, and Prototype patterns.

Structural Patterns:

- ▶ focus on the composition of classes or objects, helping to form larger structures and interfaces from simpler ones.
- ▶ Examples: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, and Proxy patterns.

Behavioral Patterns:

- ▶ define the ways in which objects interact and communicate with each other, providing solutions for the delegation of responsibilities and collaboration between objects.
- ▶ Examples: Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, and Visitor patterns.

Claimed advantages

- ▶ Re-usability
- ▶ Avoid redundancy
- ▶ Easy maintenance
- ▶ Modularity
- ▶ Increased Productivity
- ▶ Lower costs
- ▶ Higher-quality software
- ▶ Security (data hiding)

Disadvantages: what people complain about

- ▶ Size of the programs
- ▶ Effort
- ▶ Speed
- ▶ Not suitable for all types of problems
- ▶ Steep learning curve

Facts

- ▶ OOP scales; it is widely used in the industry;
- ▶ Not all languages that are advertised as OOP languages support all the features above (e.g., C++ does not have GC);
- ▶ Sometimes, misunderstanding of the OOP concepts may complicate your code.

Resources

- ▶ Chapter 10 in **Programming Languages: Principles and Paradigms** by Maurizio Gabbrielli and Simone Martini.
URL: <https://link.springer.com/book/10.1007/978-1-84882-914-5>